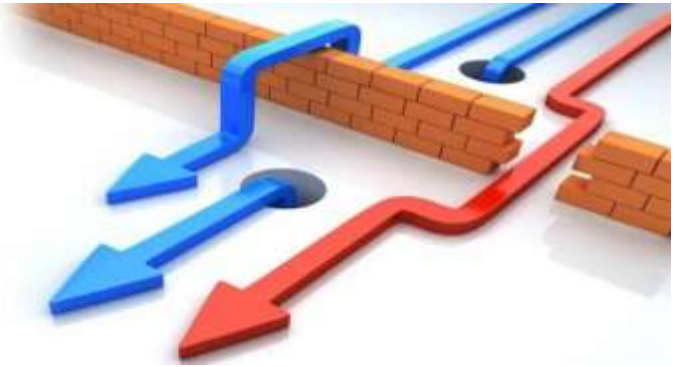


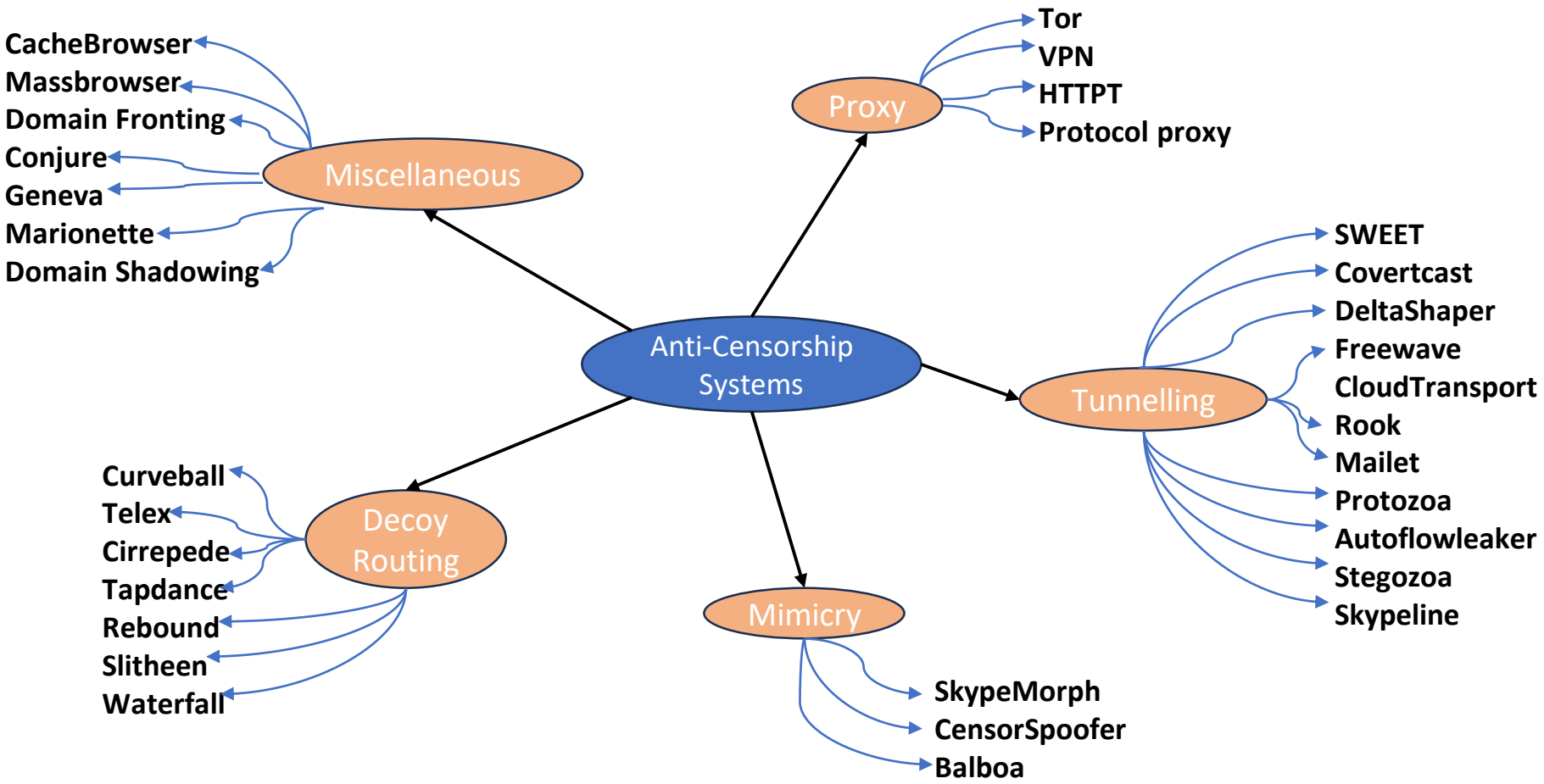
SiegeBreaker: An SDN Based Practical Decoy Routing System

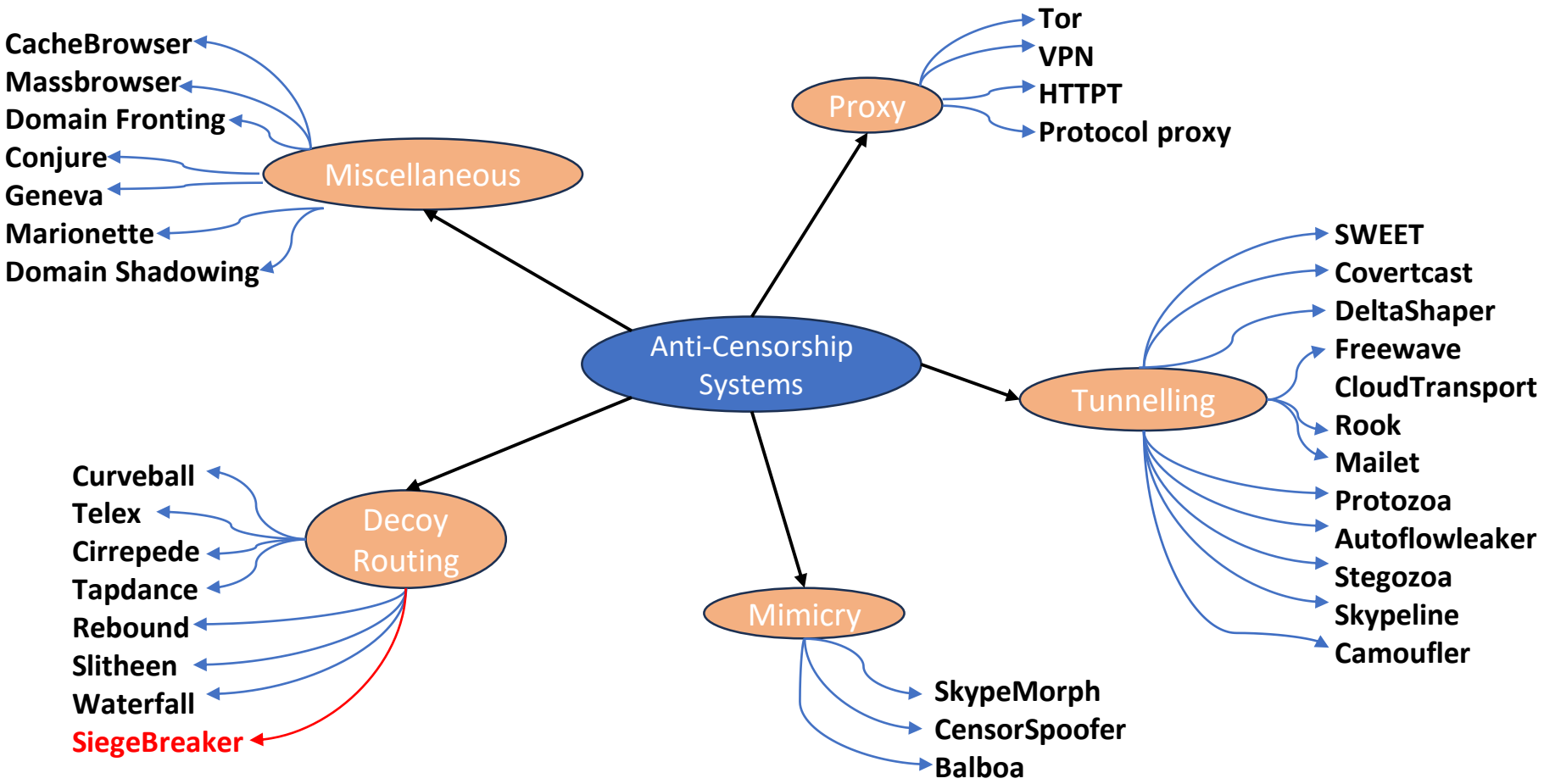
Piyush Kumar Sharma, **Devashish Gosain**, Himanshu Sagar, Chaitanya Kumar, Aneesh Dogra, Vinayak Naik, H.B. Acharya and Sambuddho Chakravarty

IIIT Delhi, India

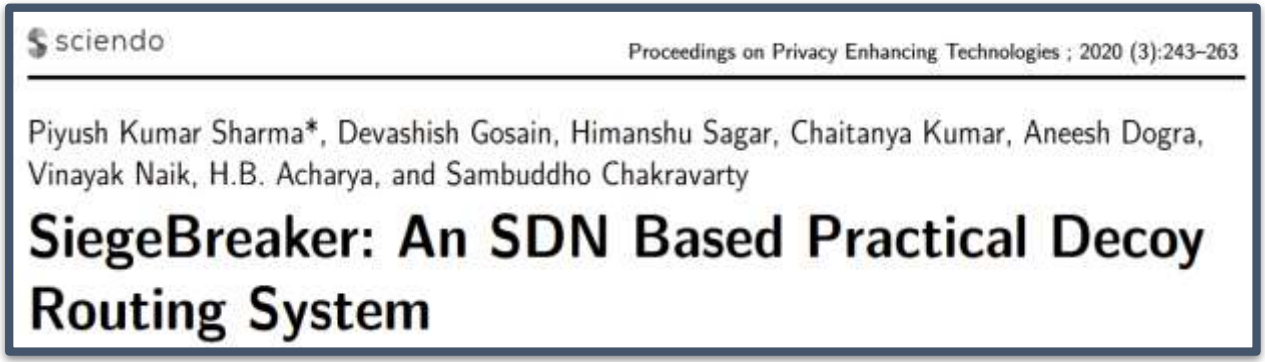
A solution for covert communication




Anti-Censorship Systems
Miscellaneous
CacheBrowser
Massbrowser
Domain Fronting
Conjure
Geneva
Marionette
Domain Shadowing
Proxy
Tor
VPN
HTTPT
Protocol proxy
Tunnelling
SWEET
Covertcast
DeltaShaper
Freewave
CloudTransport
Rook
Mailet
Protozoa
Autoflowleaker
Stegozoa
Skypeline
Decoy Routing
Curveball
Telex
Cirrepede
Tapdance
Rebound
Slitheen
Waterfall
Mimicry
SkypeMorph
CensorSpoofer
Balboa


Anti-Censorship Systems
Miscellaneous
CacheBrowser
Massbrowser
Domain Fronting
Conjure
Geneva
Marionette
Domain Shadowing
Proxy
Tor
VPN
HTTPT
Protocol proxy
Tunnelling
SWEET
Covertcast
DeltaShaper
Freewave
CloudTransport
Rook
Mailet
Protozoa
Autoflowleaker
Stegozoa
Skypeline
Camoufler
Decoy Routing
Curveball
Telex
Cirrepede
Tapdance
Rebound
Slitheen
Waterfall
SiegeBreaker
Mimicry
SkypeMorph
CensorSpoofer
Balboa

Censorship Circumvention



sciendo

Proceedings on Privacy Enhancing Technologies ; 2020 (3):243–263

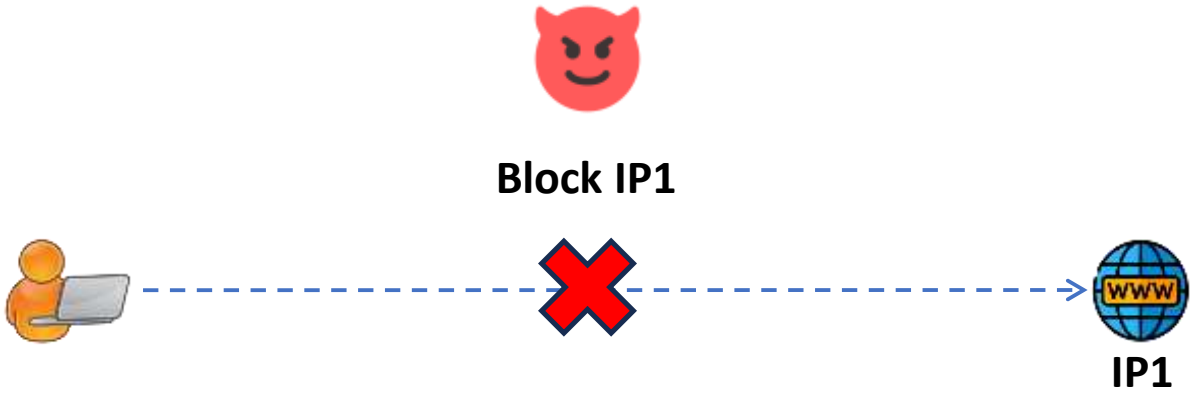
Piyush Kumar Sharma*, Devashish Gosain, Himanshu Sagar, Chaitanya Kumar, Aneesh Dogra, Vinayak Naik, H.B. Acharya, and Sambuddho Chakravarty

SiegeBreaker: An SDN Based Practical Decoy Routing System

Censorship Circumvention

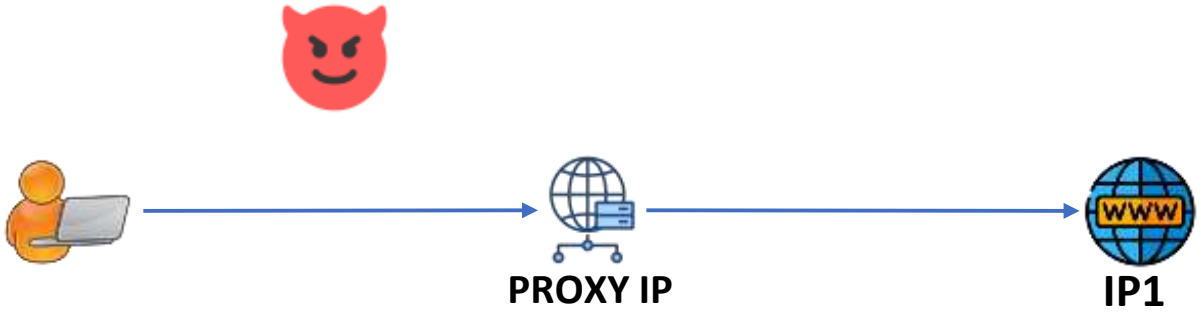


Censorship Circumvention



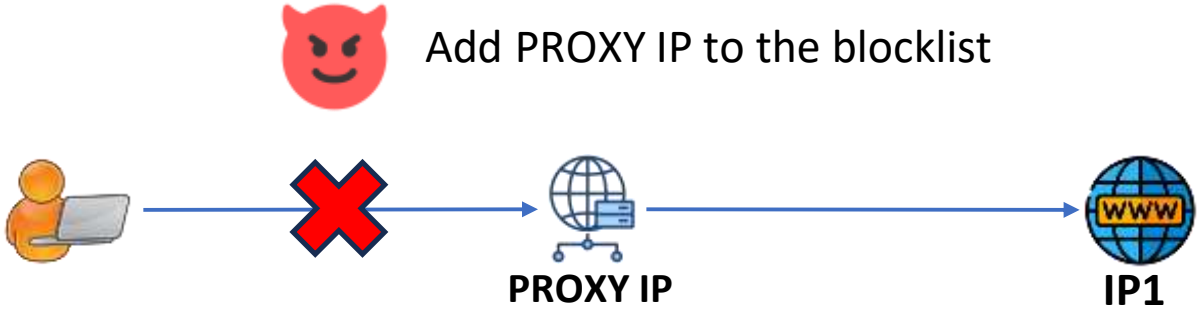
Adversary: Local Network Intermediary, e.g., ISP
Create blocklist of IPs/Domains

Censorship Circumvention



Solution: Indirect Connection
Bypasses the blocklist

Censorship Circumvention

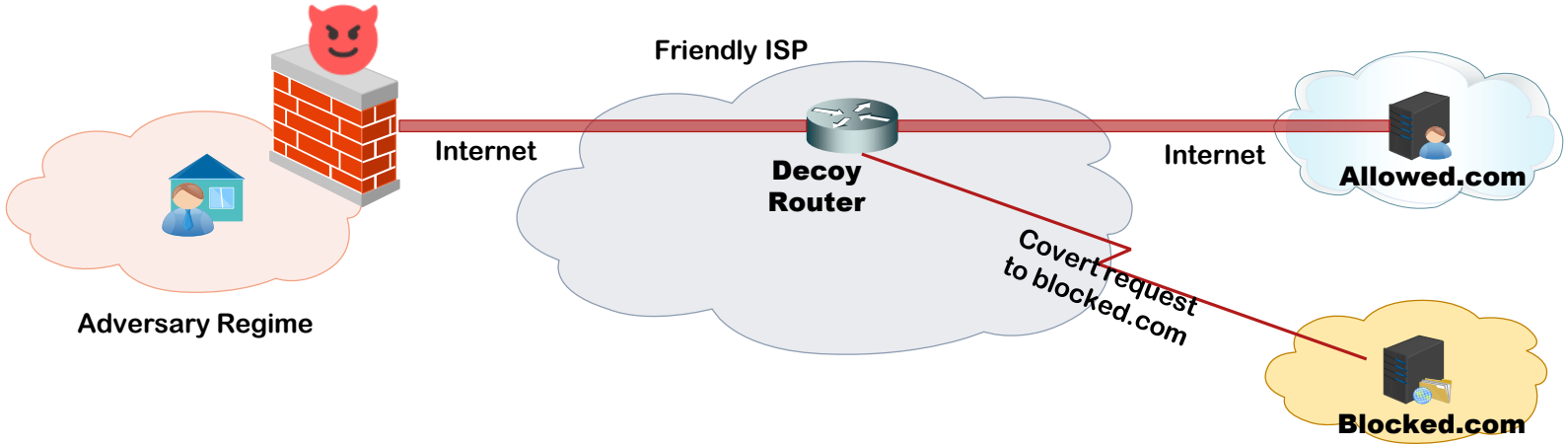




Is there a way to break out of this cat and mouse game?

Decoy Routing

- Move circumvention technology to the core of the Internet
  - Making a network router as a proxy
  - Blocking IP of router is ineffective



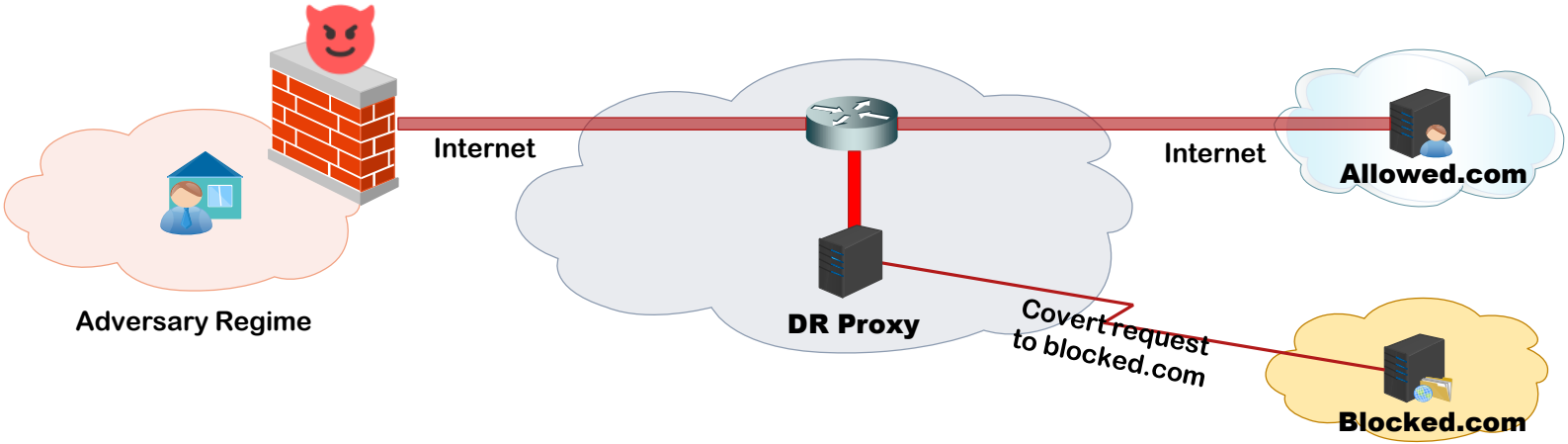
[Karlin et al. 2011, Wustrow et al. 2012, Houmansadr et. al. 2012, Wustrow et al. 2014, Ellard et al. 2015].

Research Challenges

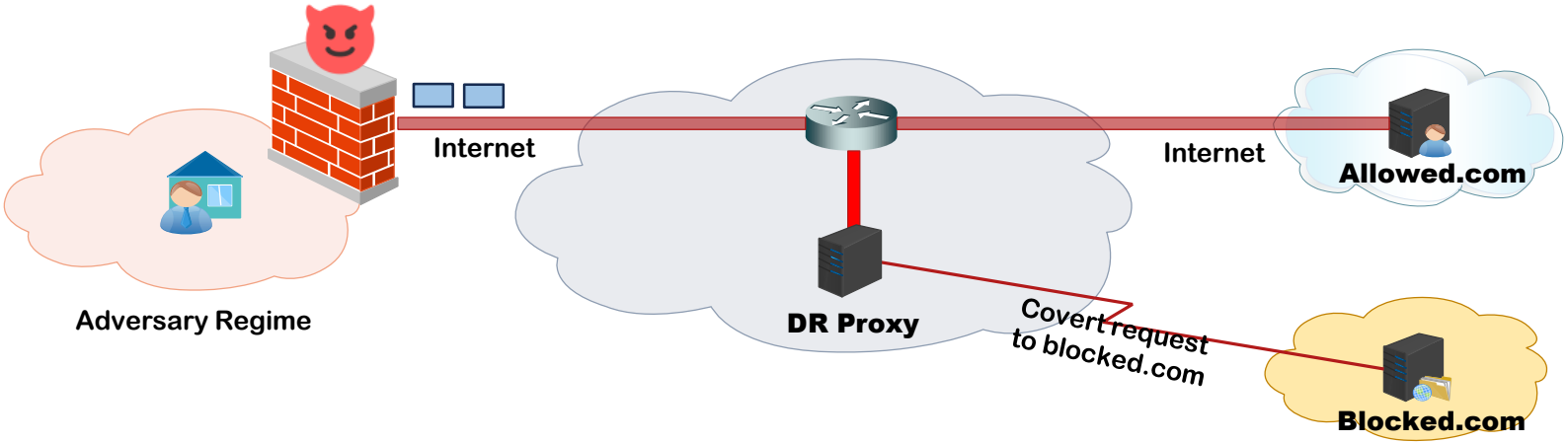
- How to build a Decoy Router?
- Covert Signaling?
- Where to place?
- Financial Incentives?
- Traffic resistance?

How to build a decoy router?

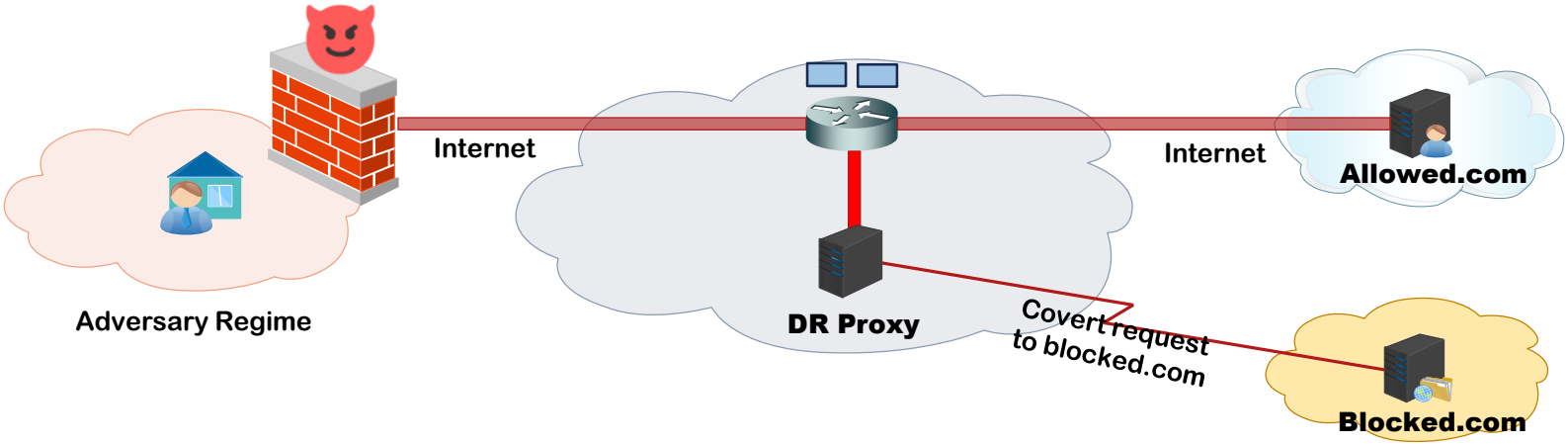
Decoy Routing: Previous Attempts



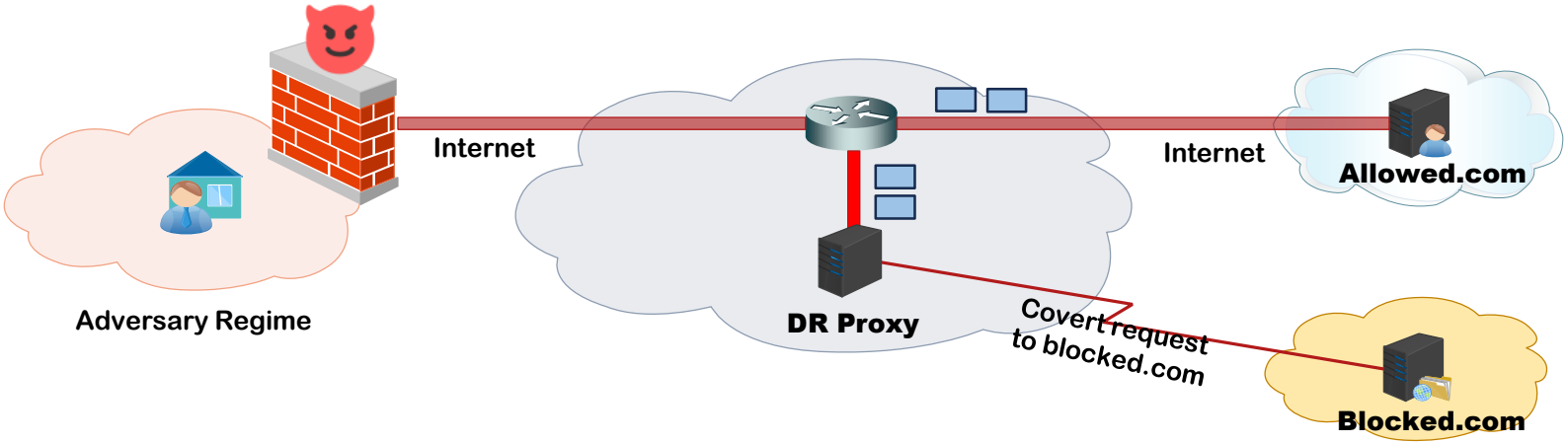
Decoy Routing: Previous Attempts



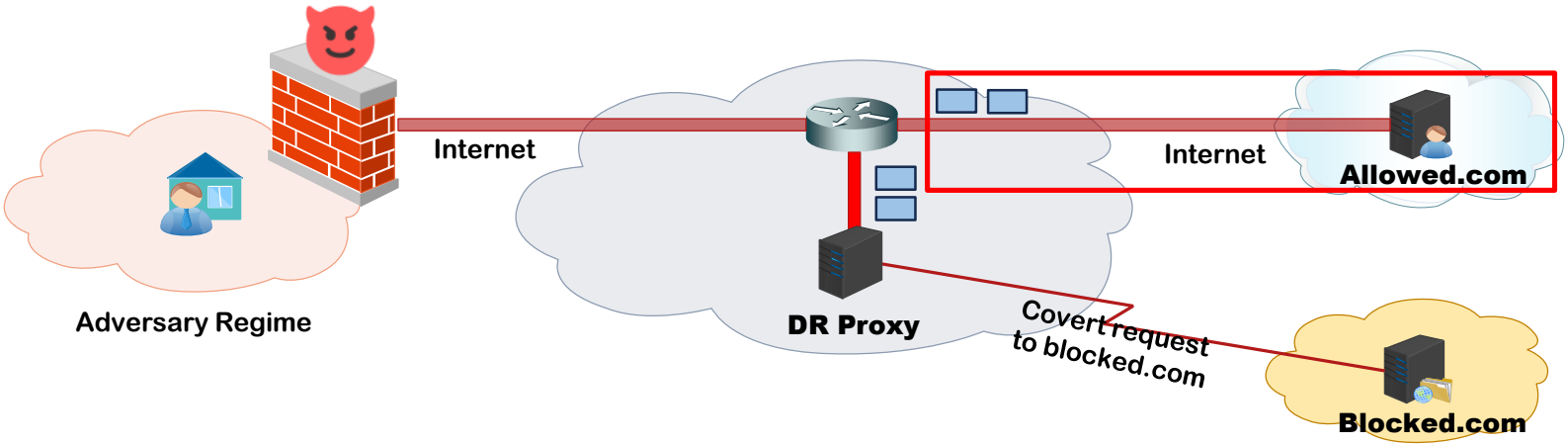
Decoy Routing: Previous Attempts



Decoy Routing: Previous Attempts



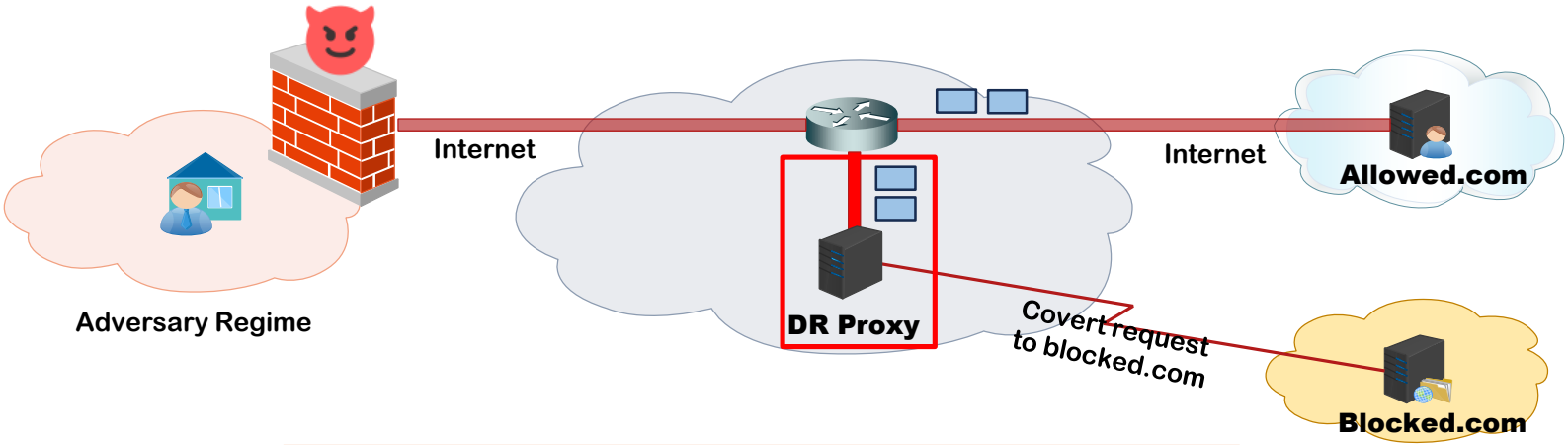
Decoy Routing: Previous Attempts



Problem: Original packets reach end server

Workaround: Send incomplete HTTP requests
Limits a single DR flow to 30s

Decoy Routing: Previous Attempts



Problem: Servers cannot handle router scale traffic

Problem: DR proxy (3rd party) observe all ISP traffic
Privacy violation of non-DR flows

Solution: SiegeBreaker

- Require on the fly reconfigurability of routers

- Need segregation of DR tasks and assign them to different entities

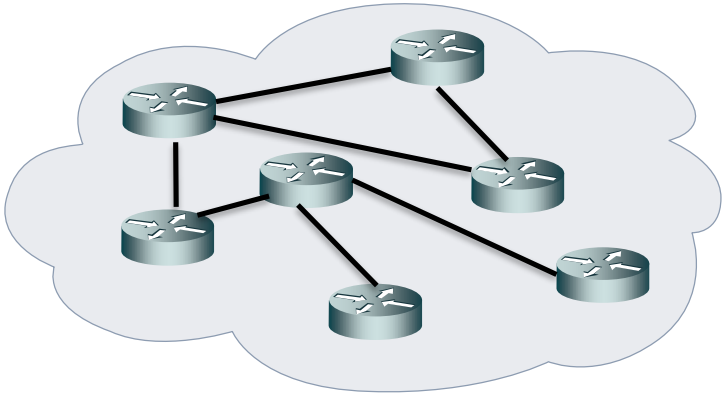
  -- Identifying DR **signalling**

  -- Processing and **forwarding packets** at line speed

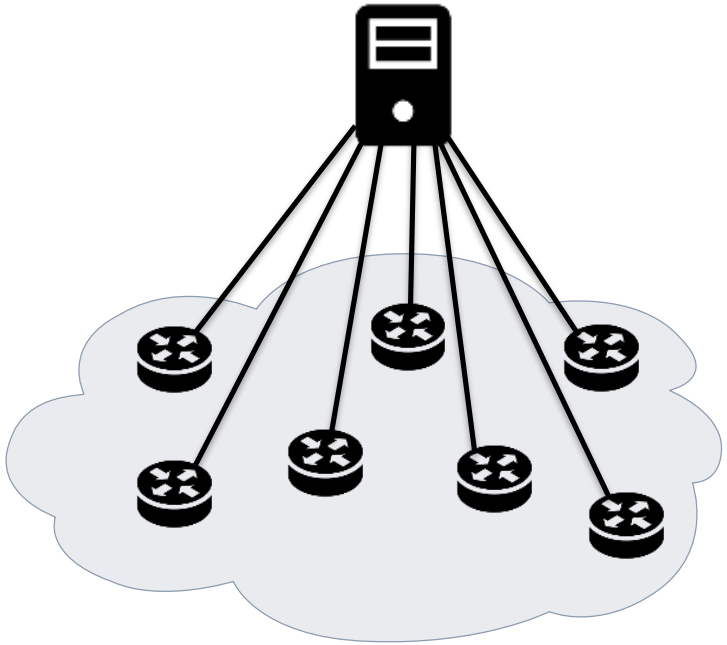
  -- **Proxy** DR flows

Programmable Software Defined Routers

SDN Architecture



Standard Networks



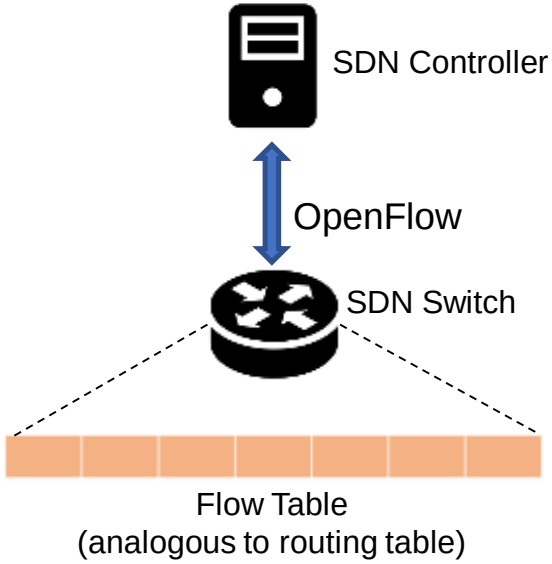
Software Defined Networks

Software Defined Network

- **SDN decouples the control and data plane**

- **Controller** (i.e., the brain)
  - Complex network operations in real time
  - Updates flow tables on SDN switches

- **SDN switches**
  - Forward packets at line rates using flow table rules



SiegeBreaker Challenges

Signalling of a decoy flow, without inspecting ISP scale traffic

Facilitating hijacking of DR flows, without alerting the censor

Reliability and performance (without OS supported TCP)

Security & Undetectability

Prevent DoS scenarios

SiegeBreaker Challenges

Signalling of a decoy flow, without inspecting ISP scale traffic

Facilitating hijacking of DR flows, without alerting the censor

Reliability and performance (without OS supported TCP)

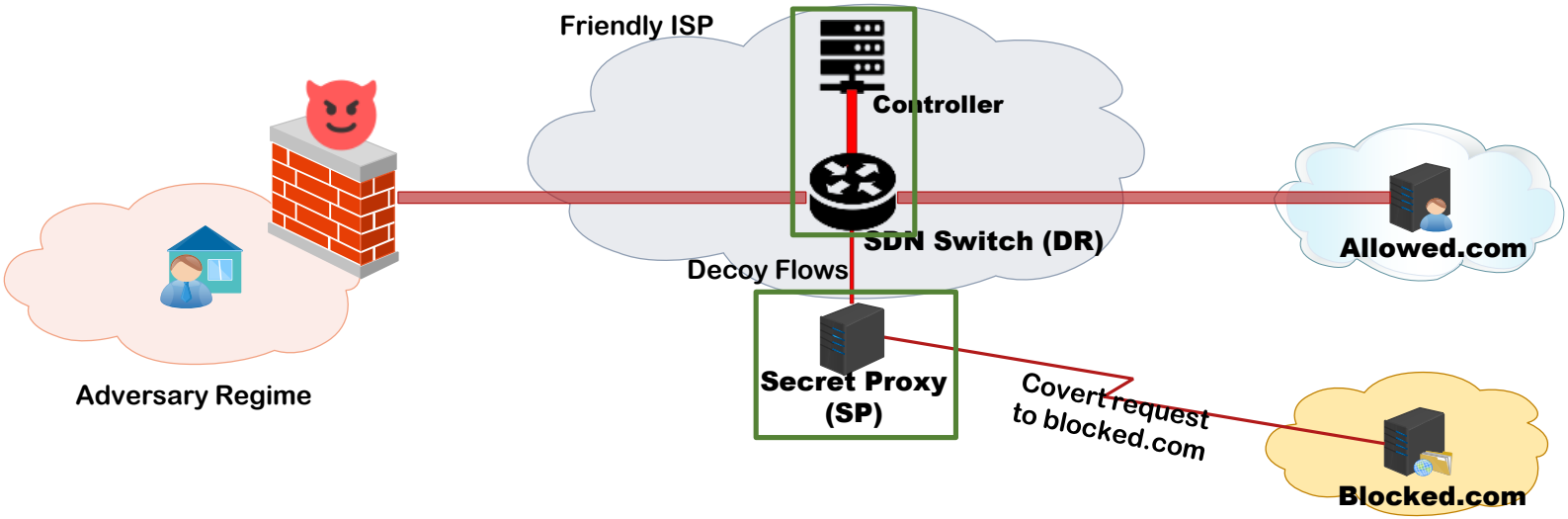
Security & Undetectability

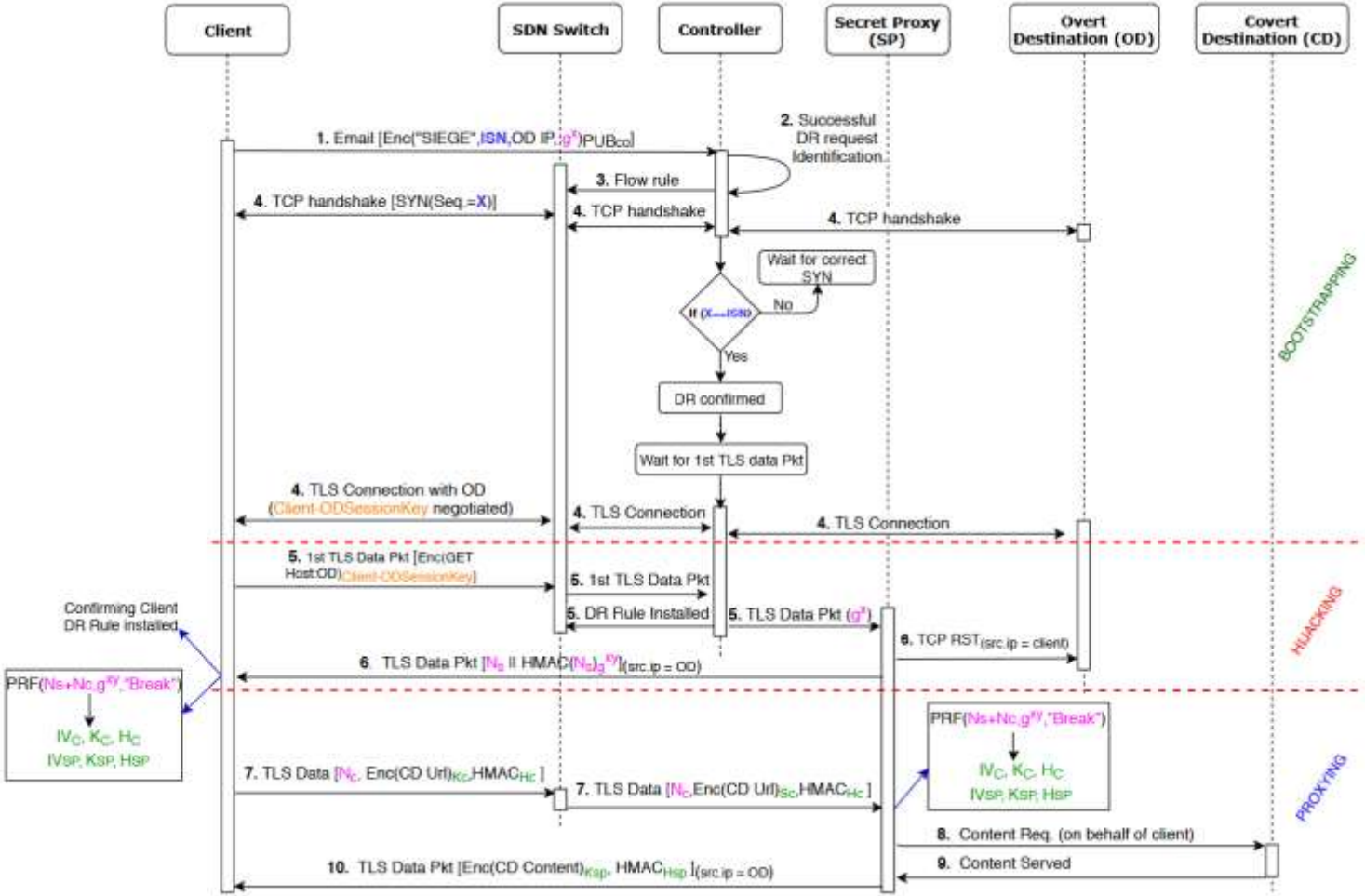
Prevent DoS scenarios

Siegebreaker Components

Controller + SDN switches belong to **the ISP**

whereas, DR/Secret proxy managed by **3rd-party maintainers**

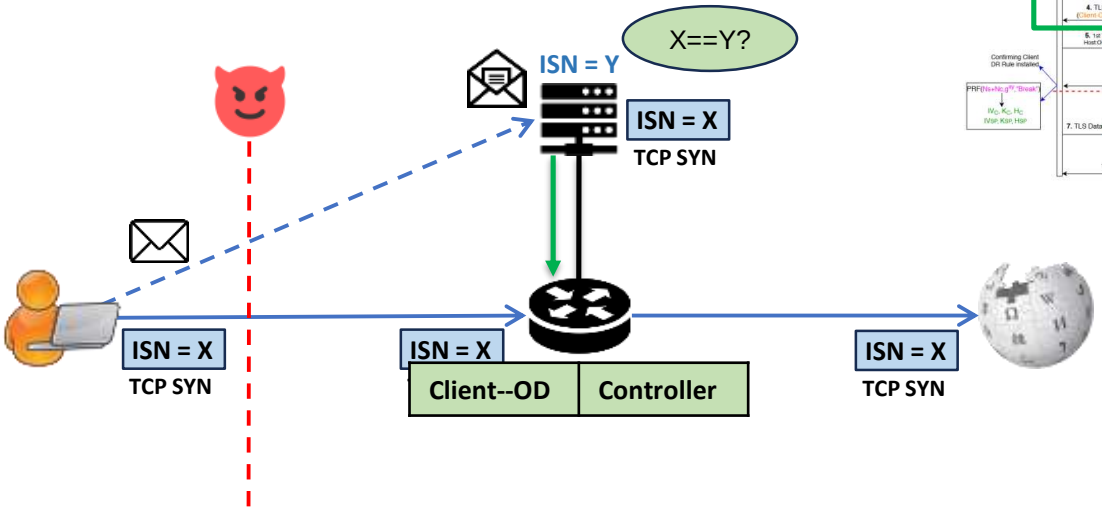


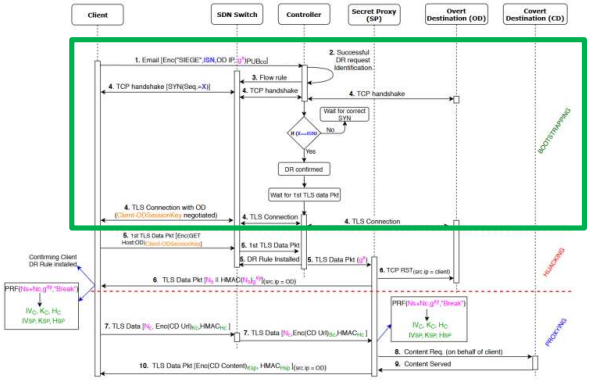

Client
SDN Switch
Controller
Secret Proxy (SP)
Overt Destination (OD)
Covert Destination (CD)
1. Email [Enc("SIEGE",ISN,OD IP,g^x)PUB_{col}]
2. Successful DR request identification
3. Flow rule
4. TCP handshake [SYN(Seq.=X)]
4. TCP handshake
4. TCP handshake
Wait for correct SYN
If (X==ISN)
No
Yes
DR confirmed
Wait for 1st TLS data Pkt
4. TLS Connection with OD (Client-ODSessionKey negotiated)
4. TLS Connection
4. TLS Connection
5. 1st TLS Data Pkt [Enc(GET Host:OD)Client-ODSessionKey]
5. 1st TLS Data Pkt
5. DR Rule Installed
5. TLS Data Pkt (g^x)
6. TCP RST(src.ip = client)
Confirming Client DR Rule installed
6. TLS Data Pkt [N_s || HMAC(N_s)$_{g^{xy}}$](src.ip = OD)
PRF(Ns+Nc,g^{xy},"Break")
IV_C, K_C, H_C
IV_{SP}, K_{SP}, H_{SP}
PRF(Ns+Nc,g^{xy},"Break")
IV_C, K_C, H_C
IV_{SP}, K_{SP}, H_{SP}
7. TLS Data [N_c, Enc(CD Url)$_{KC}$,HMAC$_{HC}$]
7. TLS Data [N_c,Enc(CD Url)$_{SC}$,HMAC$_{HC}$]
8. Content Req. (on behalf of client)
9. Content Served
10. TLS Data Pkt [Enc(CD Content)$_{Ksp}$, HMAC$_{Hsp}$](src.ip = OD)
BOOTSTRAPPING
HIJACKING
PROXYING

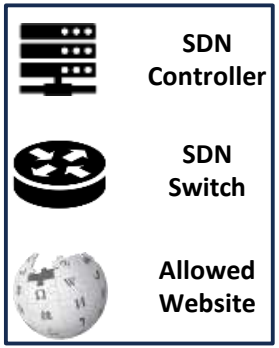
Publicly Known Information

- Public key of the controller's 2048-bit RSA public-private key-pair
- The 256-bit DH public exponent g^y of SP
- The controller's email ID

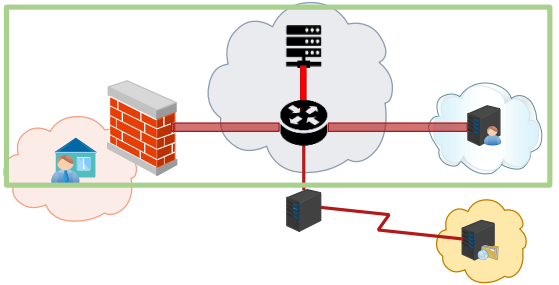
I. Bootstrapping Phase

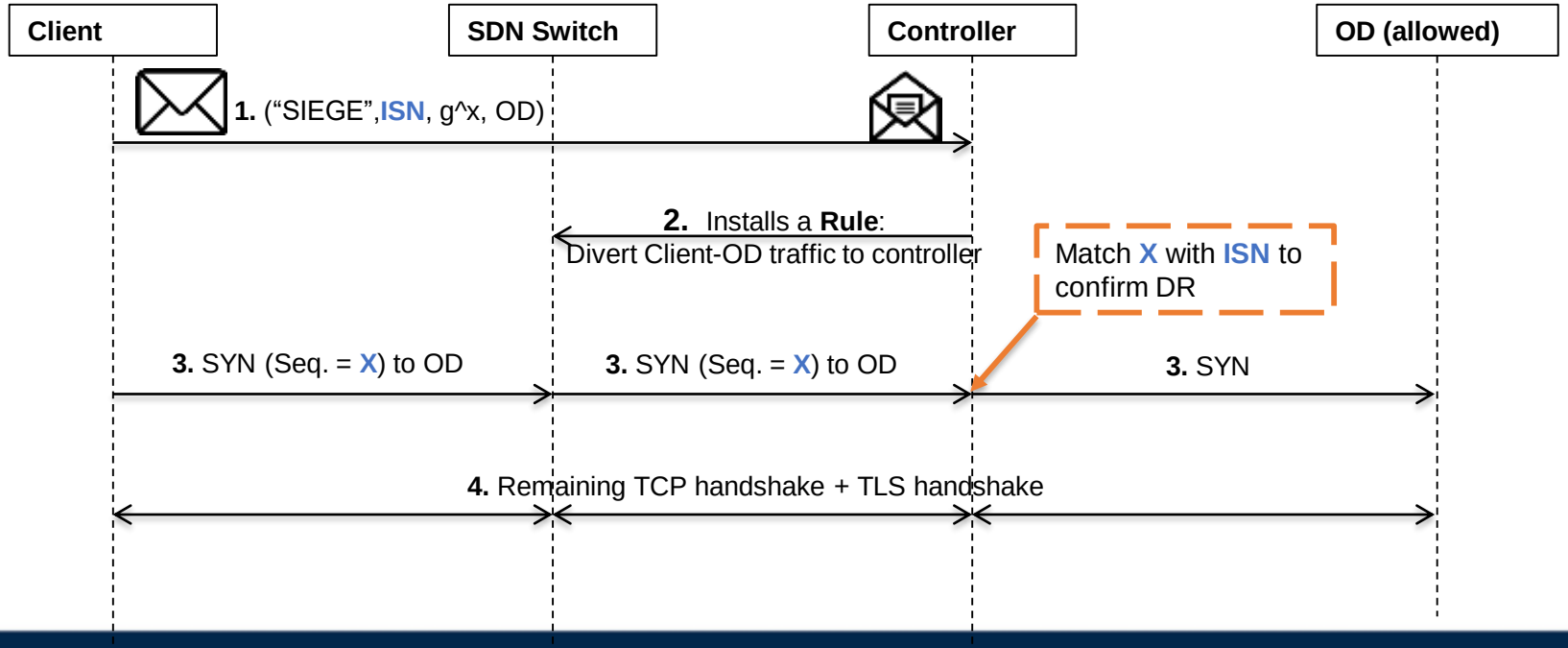




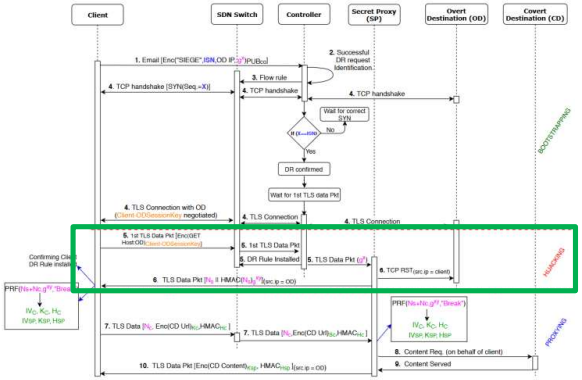


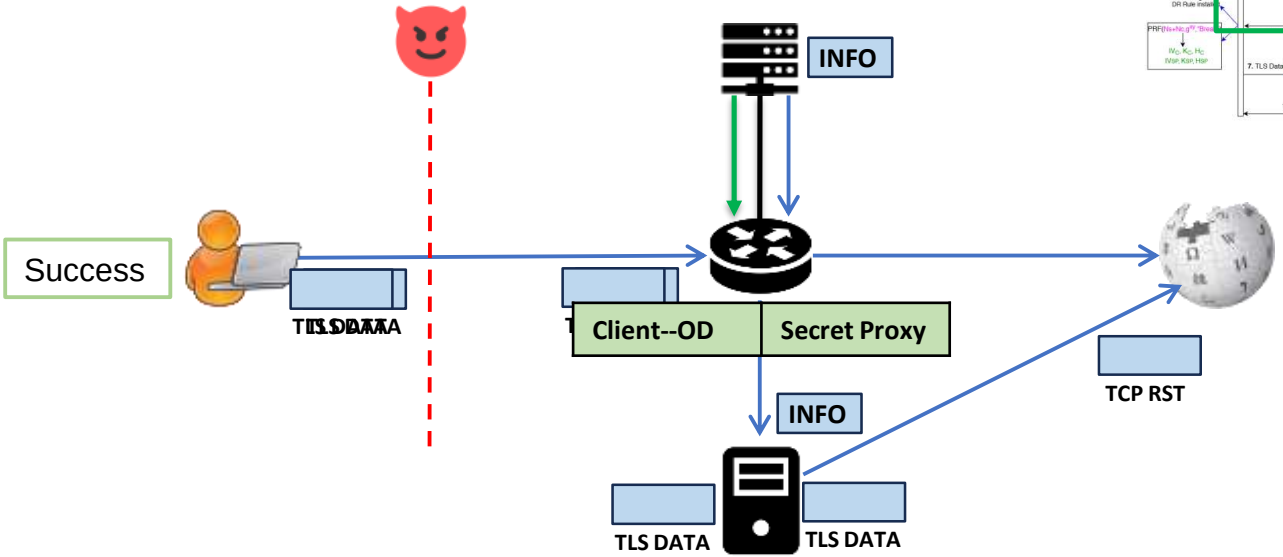
I. Bootstrapping Phase





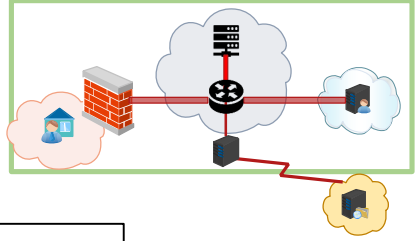
II. Hijacking Phase

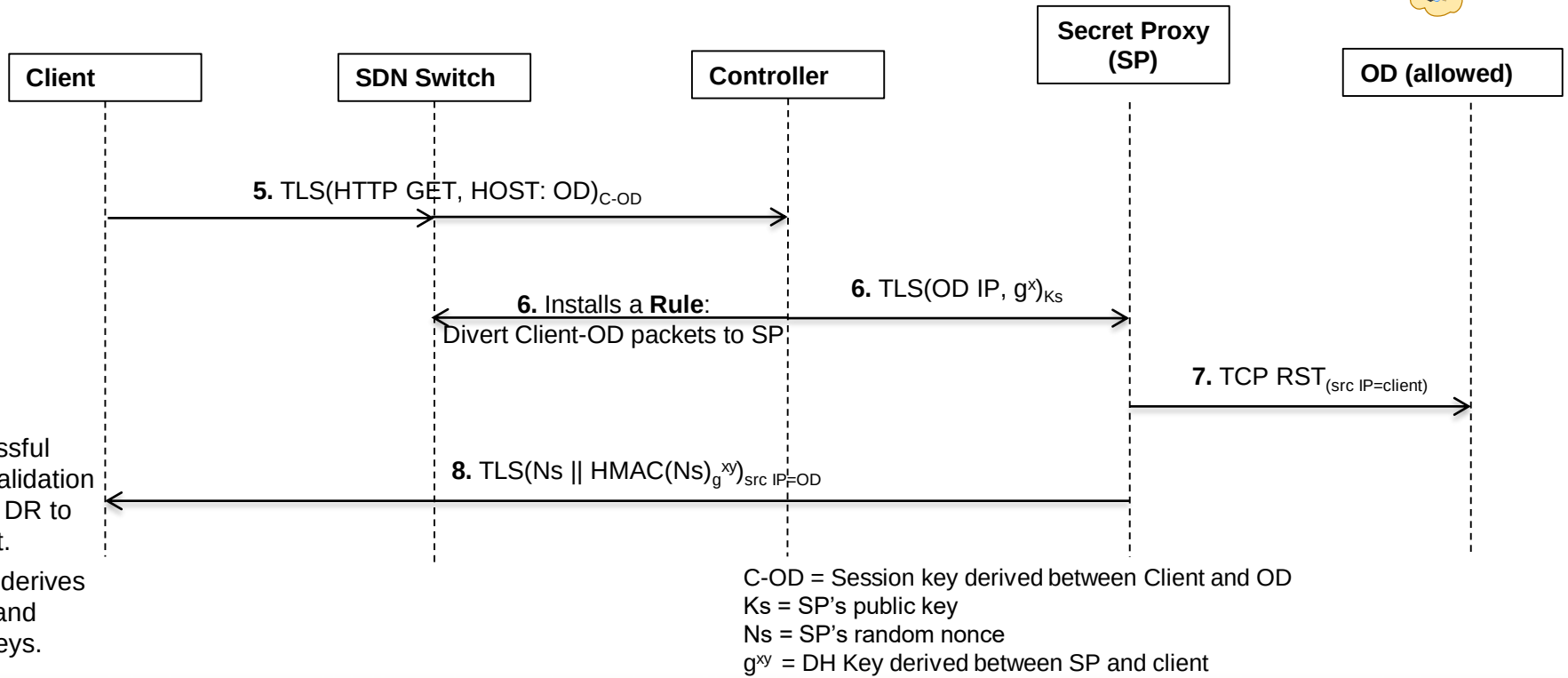






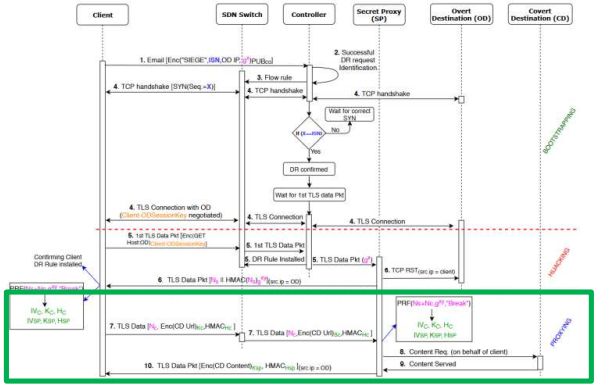
II. Hijacking Phase

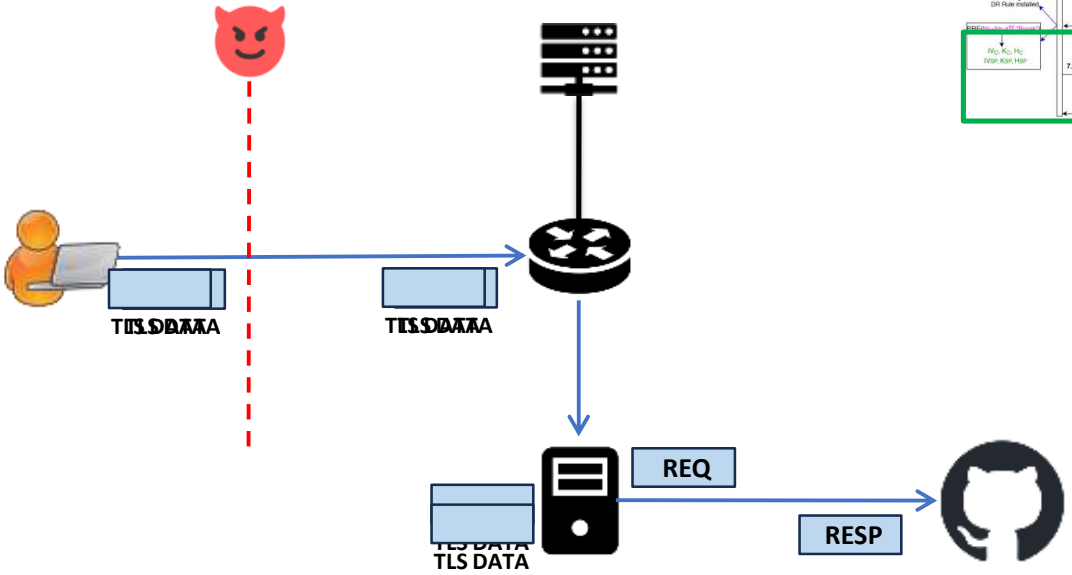


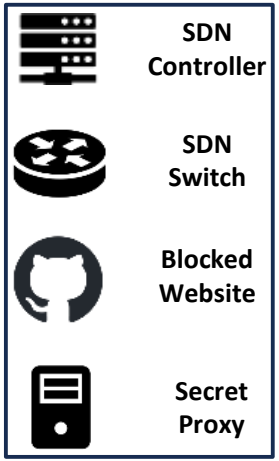


C-OD = Session key derived between Client and OD
Ks = SP's public key
Ns = SP's random nonce
g^{xy} = DH Key derived between SP and client

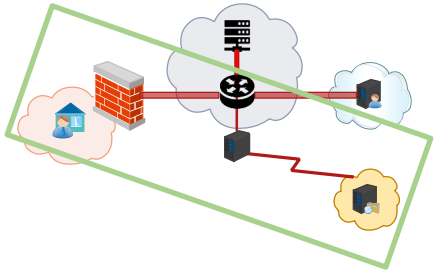
III. Proxying Phase

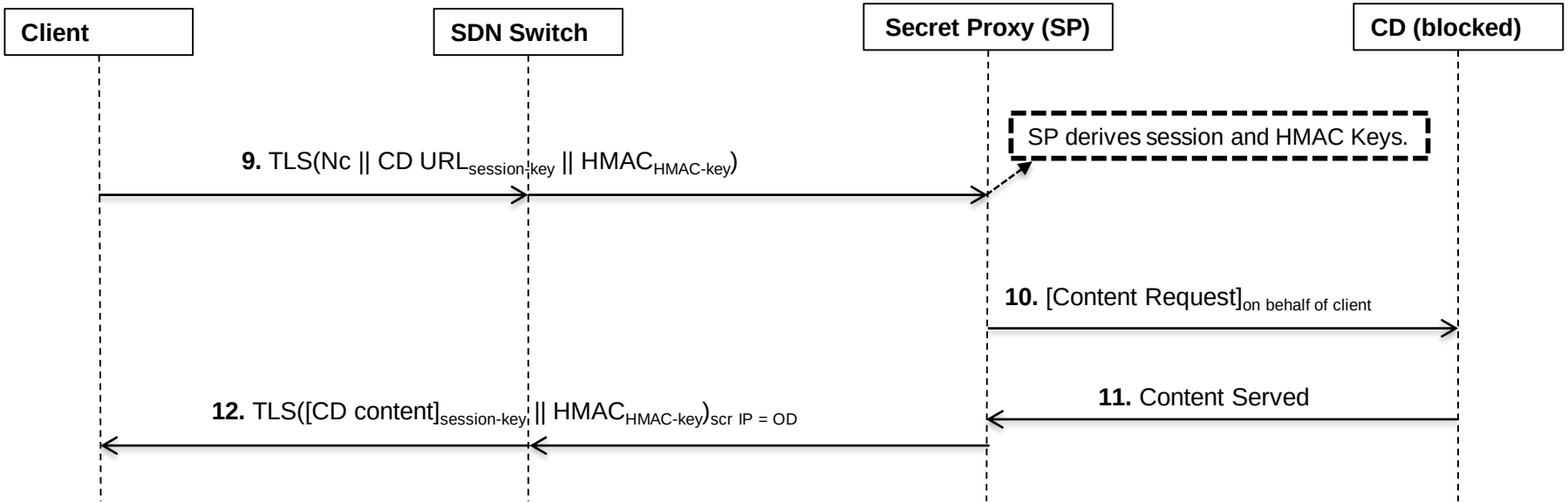






III. Proxying Phase





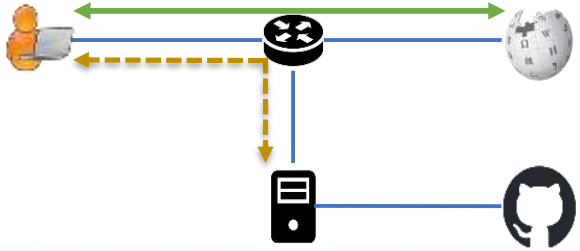
SiegeBreaker Achieved

- Segregation of decoy routing operations

Controller only inspects potential DR flows:
- using an out-of-band channel (e.g., Email)

SDN switches merely redirect them to Secret Proxy (SP)
- Efficiently at line rates

SP intercepts and proxies only the DR flows
- While implementing flow and congestion control between the client and itself



Implementation

- Test Scenario I: Mininet SDN simulator.
  - Scalability issues

- Test Scenario II: Deter testbed.
  - Do not involve hardware switches:
    - Linux machines were configured as SDN switches (using OvS)
    - Observed satisfactory performance

- Test Scenario III: a university network.
  - Used a commercial SDN switch HP3500yl.
  - Performed both in-lab and Internet based experiments.
  - Obtained promising results.

Evaluation

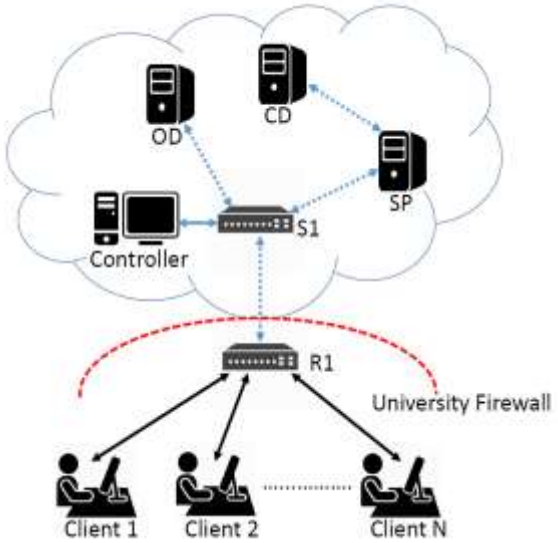
In lab experiments

1.) SB's performance in *controlled cross-traffic*
2.) Flow and congestion control of SB

Internet (deployed in university network)

1.) Accessed Alexa top-500 websites
2.) Downloaded files (<=1GB)
3.) Varying load + cross-traffic at SDN switch

Used a **commercial SDN switch** HP3500yl



In-Lab (Controlled) Experiment's Results

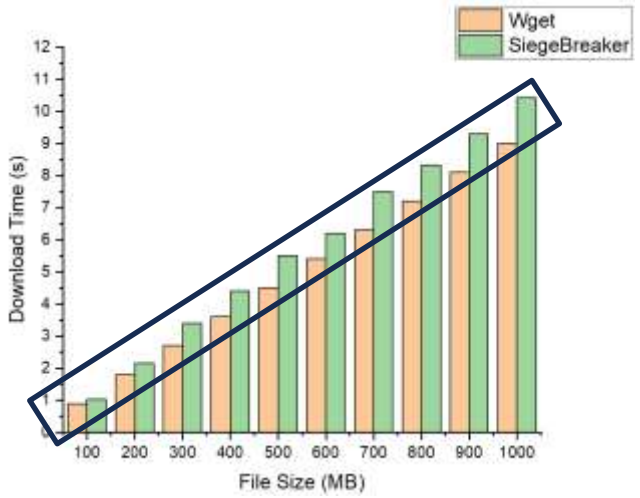


Fig. 1**. Download Time:** SB and wget downloading files with size <= 1GB

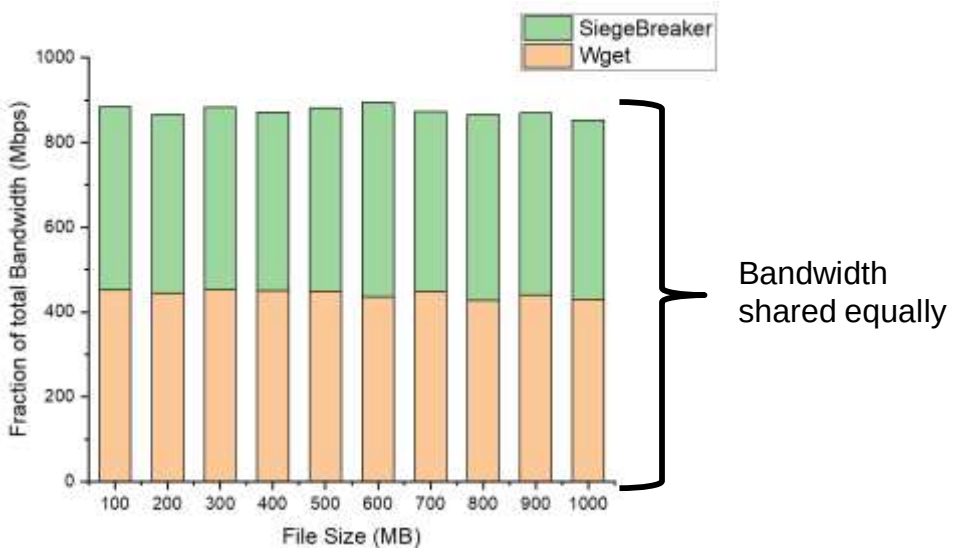


Fig. 2. **TCP fair-share:** SB and wget simultaneously downloading a file on a common link

Internet Experiment's Results

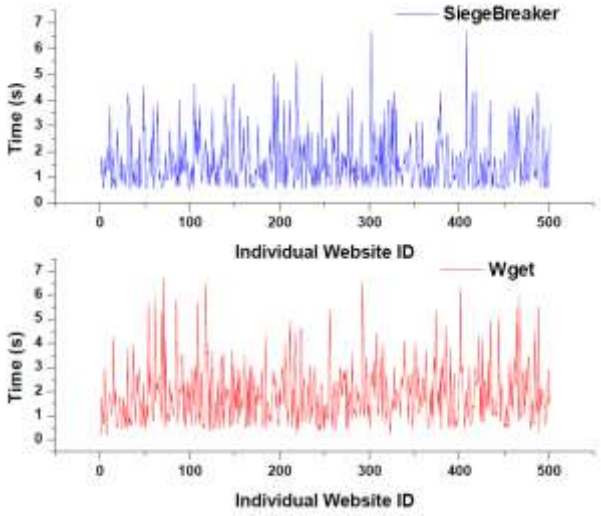


Fig. 3. **Download times for Alexa top-500 websites** accessed by 500 parallel clients of SB and wget

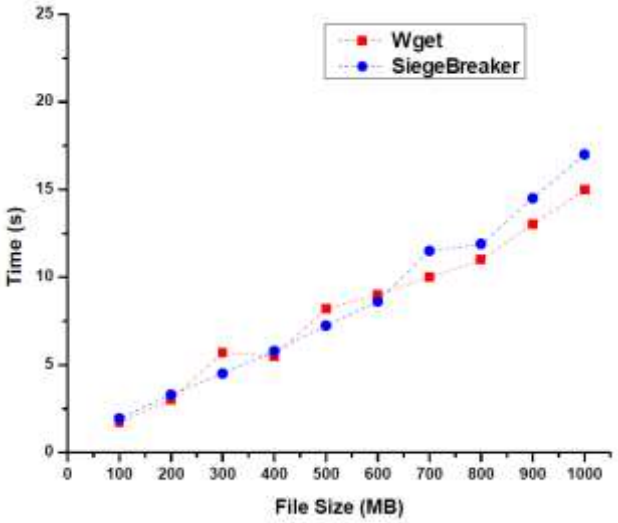


Fig. 4. **SB vs wget:** Download time for varying file sizes where file server hosted on actual Internet

Summary

Developed a practical DR system, SiegeBreaker

Segregates DR tasks

 inspects partial traffic and identifies *DR flows*

 forwards packets at *line speed*

 handles only the DR flows, thereby *preserving privacy*

https://github.com/pi-yush/SiegeBreaker

Evaluation in various in-lab and Internet experiments
First time demonstrated performance comparable to direct TCP downloads

Tested under **15GBps load** and **50k parallel flows**
A **comprehensive security analysis** demonstrating SB's resilience to attacks